Laboration 2

Laboration 2 - Länkad lista och ADT:erna kö, stack och set

Viktigt! *Som vanligt, läs "Information och regler kring laborationerna".*

Både laboration 2.1 och 2.2 ska lämnas in i Canvas. Lämna in dem så snart du känner dig färdig med dem. Du ska också lämna in en screenshot där det framgår att testfunktionen har körts utan att några asserts aktiverats.

Läs varje deluppgift noggrant då det är viktigt att du följer instruktionerna för att du ska få ihop allt rätt. Om du undrar över något eller tycker att något är otydligt, hör av dig och fråga.

Vid inmatning från användaren kan du anta att korrekt typ matas in.

Se till att vara förberedd inför labbtillfällena, så att du verkligen hinner få hjälp med det du eventuellt har problem med.

Tänk igenom hur du ska ta dig an uppgiften och planera dina lösningar innan du börjar koda – vad är funktionens uppgift och vad måste hända rent logiskt. Rita gärna upp olika scenarion för att förstå vilka problem som kan uppstå.

Laboration 2.1 – Länkad lista

Uppgift 2.1.1: Ladda ner och kompilera labbskelettet

Hämta filerna i den zippade filen LinkedList.zip som finns i Modul B i Canvas Öppna ett nytt **tomt** projekt i din utvecklingsmiljö. Lägg till filerna i projekt. Prova att kompilera, det ska gå att kompilera och köra utan fel. Det händer dock ingenting när du kör.

Uppgift 2.1.2: Bekanta dig med projektfilerna och fundera över hur listan ska fungera

Nu är det dags att välja om du vill implementera din länkade lista som enkellänkad eller dubbellänkad. När du bestämt dig, gå till `list.h` och se till att endast den ena `struct node` är avkommenterad.

Läs noga igenom `list.h` och `list.c` och fundera över hur de olika funktionerna ska fungera. Alla funktioner som är angivna i interfacet samt hjälpfunktionen `static struct node* createListNode(const Data data)` ska implementeras.

De funktioner som finns med (och måste finnas med) är:

- Skapa en ny tom lista
- Avgör om en lista är tom eller inte
- Skapa en ny listnod genom att allokera minne och sätta data (hjälpfunktion)
- Lägg till element (av typen `const Data`) först i listan
- Lägg till element (av typen `const Data`) sist i listan
- Ta bort det första elementet i listan (och frigör minnet)
- Ta bort det sista elementet i listan (och frigör minnet)
- Ta bort ett givet element (av typen `const Data`) från listan (och frigör minnet)
- Avgör om ett givet element (av typen `const Data`) finns i listan eller inte
- Räkna antalet noder i listan
- Töm hela listan (frigör minnet för alla noder)
- Skriv ut listan (här kan du anta att datat är ett heltal, `int`)
- Inför del två av laborationen ska ni även implementera funktioner som returnerar den första samt sista noden i listan (datat i noden).

Att tänka på:

- Ordet `const` framför ett argument betyder att funktionen inte kan ändra innehållet i argumentet. Detta är användbart i samband med pekare för att se till att funktionen inte ändrar på datat som pekaren pekar på.

Anledningen att typen **`const`** `Data` ska skickas in är att funktionen inte ska ändra på datat som ska ligga i listan (oavsett om det råkar vara en pekare).
- En funktion som inte ändrar på listan tar listan i form av en **`const`** `List` som argument. En funktion som ändrar på listan tar listan i form av en `List*`. ***Tänk på att*** List är en pekare till en node-typ.

- Fundera över pre- och postconditions för dina funktioner och använd asserts för att verifiera dem (i alla fall preconditions).
- Fundera på vilka specialfall du behöver kunna hantera, t.ex. att listan är tom.
- Du ska kunna fylla listan med noder, ta bort samtliga noder och sedan lägga till nya noder utan att programmet kraschar.

Du kan använda menyn i `test_list.c` för att testa din länkade lista allteftersom du färdigställer funktionerna. Om du vill får du givetvis skriva en egen menyfunktion.

Titta också igenom testfunktionen som finns i `test_list.c`.

Uppgift 2.1.3: Implementera din länkade lista

Nu är det dags att implementera. Koden ska skrivas i `list.c`. Du ska inte ändra eller lägga till/ta bort någonting från `list.h`. Tänk på i vilken ordning funktionerna implementeras samt att du testar dem allt eftersom de blir färdiga. Använd debuggern om du stöter på problem.

Tips: Använd papper och penna för att kunna rita upp vad du vill ska hända och vad som faktiskt händer när koden körs. Detta är ett sätt att skapa sig en större förståelse för koden och funktionaliteten samt hitta logiska problem.

Uppgift 2.1.4: Testa implementationen

När samtliga funktioner är implementerade och fungerar som tänkt är det dags att köra testfunktionen i `test_list.c`. Denna får inte ändras (förutom om `#define _CRT_SECURE_NO_WARNINGS` behöver läggas till).

Om en assert aktiveras behöver du ta reda på vilka fel som finns i din implementation (kommentarerna i `test_list.c` bör hjälpa). Om du inte fastnar i någon assert så fungerar den länkade listan som labbspecen säger och du kan gå vidare till uppgift 2.1.

Laboration 2.2 – ADT:er

I denna uppgift ska du göra implementationer för ADT:erna **Stack**, **Queue** och **Set**. Samtliga ska implementeras som länkade listor. Hämta filerna i den zippade filen Queue_Stack_Set.zip som finns i Modul B i Canvas. `list.h` och `list.c` från laboration 2.1 måste också ligga med i projektet, du får dock inte ändra någonting i list-filerna.

I `test_queue_stack_set.c` finns testfunktioner för var och en av ADT:erna. Dessa får inte ändras. Här finns också menyfunktioner för vardera av ADT:erna.

Om en assert i testfunktionen aktiveras behöver du ta reda på vilka fel som finns i din implementation. Om du inte fastnar i några asserts så fungerar den testade ADT:n

Här är det viktigt att återanvända kod från listan, inte kopiera eller skriva av/skriva om. Varje funktion i vardera av stacken, kön och setet kommer bli väldigt korta, enbart ett fåtal rader.